

```

class FakeStripeAPI:
    def __init__(self):
        self.charges = []
        self.idempotency_keys = set()
    def charge(self, amount, idempotency_key=None):
        if idempotency_key and idempotency_key in self.idempotency_keys:
            print(f"[STRIPE] Idempotent hit for {idempotency_key}. No charge.")
            return {"status": "success", "cached": True}
        print(f"[STRIPE] Charging ${amount}...")
        self.charges.append(amount)
        if idempotency_key:
            self.idempotency_keys.add(idempotency_key)
        return {"status": "success"}

```

```

class FakeDatabase:
    def __init__(self):
        self.processed_events = set()
        self.payments = {}
    def commit(self):
        print("[DB] Transaction committed.")
    def rollback(self):
        print("[DB] Transaction rolled back.")

```

```

class FixedPaymentService:
    def __init__(self, db, stripe):
        self.db = db
        self.stripe = stripe
    def process_payment(self, command_id, amount):
        print(f"\n--- Processing {command_id} ---")

        if command_id in self.db.processed_events:
            status = self.db.payments.get(command_id)
            if status == "IN_PROGRESS":
                print("[DB] Found IN_PROGRESS. Re-verifying with Stripe...")
                idempotency_key = f"IDEM-{command_id}"
                self.stripe.charge(amount, idempotency_key=idempotency_key)
                self.db.payments[command_id] = "COMPLETED"
                self.db.commit()
                return
            elif status == "COMPLETED":
                print("[DB] Already completed. Skipping.")
                return

```

# Phase 1: Local Intent Reservation

```

        self.db.processed_events.add(command_id)
        self.db.payments[command_id] = "IN_PROGRESS"
        self.db.commit()

    # Phase 2: External API with Idempotency Key
    idempotency_key = f"IDEM-{command_id}"
    self.stripe.charge(amount, idempotency_key=idempotency_key)

    # Simulate crash
    raise Exception("CRASH: DB connection dropped before final commit!")

if __name__ == "__main__":
    db = FakeDatabase()
    stripe = FakeStripeAPI()
    service = FixedPaymentService(db, stripe)

    # Kafka delivery 1
    try:
        service.process_payment("CMD-PAY-101", 10.00)
    except Exception as e:
        print(e)
        db.rollback()

    # Kafka redelivery
    try:
        service.process_payment("CMD-PAY-101", 10.00)
    except Exception as e:
        print(e)
        db.rollback()

    print(f"\nTotal charges: ${sum(stripe.charges)}")

```